

CA - Experiment 3 Part 1 - Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation

Here's the **full step-by-step guide** for working with **Gradle in IntelliJ IDEA**, including **setup, project creation, website deployment, Selenium testing, and JAR packaging**.

Gradle in IntelliJ IDEA: A Complete Guide

1 Introduction to Gradle

Gradle is a powerful build automation tool used for **Java, Kotlin, and Android projects**. It provides **fast builds, dependency management, and flexibility** using Groovy/Kotlin-based scripts.

♦ Why Use Gradle?

- **Incremental builds** (faster execution)
- **Dependency management** (supports Maven/Ivy)
- **Customizable tasks**
- **Multi-project support**

2 Installing and Setting Up Gradle in IntelliJ IDEA (Already Covered In Experiment 1)

Step 1: Install Gradle

- **Windows** (using Chocolatey):

```
1 choco install gradle
2
```

- **macOS** (using Homebrew):

```
1 brew install gradle
2
```

- **Linux:**

```
1 sdk install gradle
2
```

Step 2: Verify Installation

```
1 gradle -v
2 gradle --version
```

This should display the Gradle version.

3 Creating a Gradle Project in IntelliJ IDEA

Step 1: Open IntelliJ IDEA and Create a New Project

1. Click on **"New Project"**.
2. Select **"Gradle"** (under Java/Kotlin).
3. Choose **Groovy or Kotlin DSL (Domain Specific Language)** for the build script.
4. Set the **Group ID** (e.g., `com.example`).
5. Click **Finish**.

Step 2: Understanding Project Structure

```
1 my-gradle-project
2 |— build.gradle (Groovy Build Script)
3 |— settings.gradle
4 |— src
5 |   |— main
6 |   |   |— java
7 |   |   |— resources
8 |   |— test
9 |   |   |— java
10 |   |   |— resources
11
```

4 Build and Run a Simple Java Application

Step 1: Modify `build.gradle` (Groovy DSL)

```
1 plugins {
2     id 'application'
3 }
4
5 repositories {
6     mavenCentral()
7 }
8
9 dependencies {
10     testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'
11 }
12
13 application {
14     mainClass = 'com.example.Main'
15 }
16
```

Step 2: Create `Main.java` **in** `src/main/java/com/example`

```

1 package com.example;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println("Hello from Gradle!");
6     }
7 }
8

```

Step 3: Build and Run the Project

- In **IntelliJ IDEA**, open the **Gradle tool window** (View → Tool Windows → Gradle).
- Click `Tasks > application > run`.
- Or run from terminal:

```

1 gradle run
2

```

5 Hosting a Static Website on GitHub Pages**Step 1: Create a** `/docs` **Directory**

- Create `docs` inside the **root folder** (not in `src`).
- Add your **HTML, CSS, and images** inside `/docs`.

Step 2: Modify `build.gradle` **to Copy Website Files (This is optional)**

```

1 task copyWebsite(type: Copy) {
2     from 'src/main/resources/website'
3     into 'docs'
4 }
5

```

Step 3: Commit and Push to GitHub

```

1 git add .
2 git commit -m "Deploy website using Gradle"
3 git push origin main
4

```

Step 4: Enable GitHub Pages

- Go to **GitHub Repo → Settings → Pages**.
- Select the `/docs` **folder** as the source.

Your website will be hosted at:

```

1 https://yourusername.github.io/repository-name/
2

```

6 Testing the Website using Selenium & TestNG in IntelliJ IDEA

Step 1: Add Selenium & TestNG Dependencies in `build.gradle`

```
1 dependencies {
2     testImplementation 'org.seleniumhq.selenium:selenium-java:4.28.1' // use the latest stable
    version
3     testImplementation 'org.testng:testng:7.4.0' // use the latest stable version
4 }
5
6 test {
7     useTestNG()
8 }
9
```

Step 2: Write a Test Script (`src/test/java/org/test/WebpageTest.java`)

```
1 package org.test;
2
3 import org.openqa.selenium.WebDriver;
4 import org.openqa.selenium.chrome.ChromeDriver;
5 import org.testng.Assert;
6 import org.testng.annotations.AfterTest;
7 import org.testng.annotations.BeforeTest;
8 import org.testng.annotations.Test;
9
10 import static org.testng.Assert.assertTrue;
11
12 public class WebpageTest {
13     private static WebDriver driver;
14
15     @BeforeTest
16     public void openBrowser() throws InterruptedException {
17         driver = new ChromeDriver();
18         driver.manage().window().maximize();
19         Thread.sleep(2000);
20         driver.get("https://sauravsarkar-codersarcade.github.io/CA-GRADLE/");
21     }
22
23     @Test
24     public void titleValidationTest(){
25         String actualTitle = driver.getTitle();
26         String expectedTitle = "Tripillar Solutions";
27         Assert.assertEquals(actualTitle, expectedTitle);
28         assertTrue(true, "Title should contain 'Tripillar'");
29     }
30
31     @AfterTest
32     public void closeBrowser() throws InterruptedException {
33         Thread.sleep(1000);
34         driver.quit();
35     }
36 }
37
38
39 }
40
```

Step 3: Run the Tests

- Open the **Gradle tool window** in IntelliJ.
- Click **Tasks > verification > test**. **“Recommended”**
- Or run from terminal:

```
1 gradle test // Fails sometimes due to terminal issues
2
```

7 Packaging a Gradle Project as a JAR

Step 1: Modify `build.gradle` for JAR Packaging

```
1 plugins {
2     id 'java'
3     id 'application'
4 }
5
6 application {
7     mainClass = 'com.example.Main'
8 }
9 jar {
10     manifest {
11         attributes 'Main-Class': 'com.example.Main' // This tells Java where to start execution
12     }
13 }
14
```

Step 2: Build and Package the JAR

```
1 gradle jar
2
```

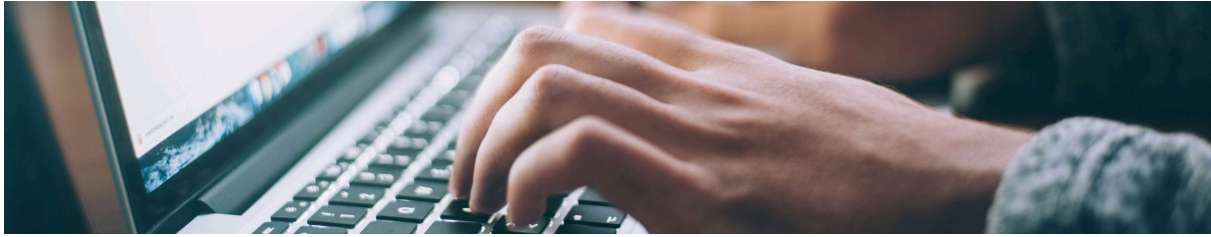
The JAR file will be generated in `build/libs/`.

Step 3: Run the JAR

```
1 java -jar build/libs/<my-gradle-project>.jar
2
3 Expected output:
4 Hello from Gradle!
5
```

8 Gradle Lifecycle & Common Commands

Task	Command	Description
Initialize Project	<code>gradle init</code>	Creates a new Gradle project.
Compile Code	<code>gradle build</code>	Compiles the project.
Run Application	<code>gradle run</code>	Runs the application.



CA - Experiment 3 Part 2 : GRADLE KOTLIN DSL WORKFLOW || IntelliJ Idea

Here's a **detailed and well-formatted** documentation Gradle Kotlin DSL setup, including explanations, code snippets, and color-coded syntax (for reference when viewing in an IDE).

⚠ Important Note : This experiment is only required if **students** or **VTU** asks you to show **GRADLE** with **KOTLIN DSL**, or else you can skip this.

Gradle Kotlin DSL: Setting Up & Building a Kotlin Project in IntelliJ IDEA

1 Setting Up the Gradle Project

Step 1: Create a New Project

1. Open **IntelliJ IDEA**.
2. Click on **File > New > Project**.
3. Select **Gradle** as the build system.
4. Choose **Kotlin** as the language.
5. Select **Gradle Kotlin DSL** (it will generate `build.gradle.kts`).
6. Name your project (e.g., `MVNGRDKOTLINDEMO`).
7. Set the **JDK** (use **JDK 17.0.4**, since that's your version).
8. Click **Finish**.

2 Understanding `build.gradle.kts`

After creating the project, the default `build.gradle.kts` file looks like this:

```
1 import org.jetbrains.kotlin.gradle.tasks.KotlinCompile
2
3 plugins {
4     kotlin("jvm") version "1.8.10" // Use latest stable Kotlin version
5     application
6 }
7
8 group = "org.example"
9 version = "1.0-SNAPSHOT"
10
11 repositories {
12     mavenCentral()
```

```

13 }
14
15 dependencies {
16     implementation(kotlin("stdlib")) // Kotlin Standard Library
17     testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.2")
18     testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.2")
19 }
20
21 tasks.test {
22     useJUnitPlatform()
23 }
24
25 tasks.withType<KotlinCompile> {
26     kotlinOptions.jvmTarget = "17" // Match with your JDK version
27 }
28
29 application {
30     mainClass.set("MainKt") // Update this if using a package
31 }
32

```

3 Creating the Main Kotlin File

Now, create your `Main.kt` file inside `src/main/kotlin/`.

If you're using a package (e.g., `org.example`), it should look like:

```

1 package org.example
2
3 fun main() {
4     println("Hello, Gradle with Kotlin DSL!")
5 }
6

```

If you're **not** using a package, remove the `package` line and ensure `mainClass.set("MainKt")` in `build.gradle.kts`.

4 Building and Running the Project

Build the Project

```

1 ./gradlew build
2

```

Run the Project

```

1 ./gradlew run
2

```

5 Packaging as a JAR

To run the project without IntelliJ, we need a **JAR file**.

Step 1: Create a Fat (Uber) JAR

Modify `build.gradle.kts`:

```

1 tasks.register<Jar>("fatJar") {
2     archiveClassifier.set("all")
3     duplicatesStrategy = DuplicatesStrategy.EXCLUDE
4     manifest {
5         attributes["Main-Class"] = "MainKt"
6     }
7     from(configurations.runtimeClasspath.get().map { if (it.isDirectory) it else zipTree(it) })
8     with(tasks.jar.get() as CopySpec)
9 }
10

```

Step 2: Build the Fat JAR

```

1 ./gradlew fatJar
2

```

Step 3: Run the Fat JAR

```

1 java -jar build/libs/MVNGRDKOTLINDEMO-1.0-SNAPSHOT-all.jar
2

```

6 Common Gradle Commands

Command	Description
<code>./gradlew build</code>	Builds the project
<code>./gradlew run</code>	Runs the application
<code>./gradlew test</code>	Runs the tests
<code>./gradlew clean</code>	Cleans the build directory
<code>./gradlew fatJar</code>	Creates a runnable JAR

7 Additional Features

COMMIT TO ACHIEVE

Adding Custom Gradle Tasks

Example: A simple task that prints "Hello, Gradle!"

```

1 tasks.register("hello") {
2     doLast {
3         println("Hello, Gradle!")
4     }
5 }
6

```

Run it with:

```

1 ./gradlew hello
2

```


8 Troubleshooting & Fixes

1. `java.lang.NoClassDefFoundError: kotlin/jvm/internal/Intrinsics`

✓ **Fix:** Use a Fat JAR (`fatJar`) to include Kotlin dependencies.

2. `mainClass` **Not Found**

✓ **Fix:** Ensure `mainClass.set("MainKt")` or use the correct package (`org.example.MainKt`).

3. Gradle Wrapper Missing

✓ **Fix:** Generate it using:

```
1 gradle wrapper
2
```

✓ **You're all set!** 🚀

This guide ensures you have a **fully working** Gradle project with Kotlin DSL, dependency management, custom tasks, and a runnable JAR. 😊



CODERS ARCADE

COMMIT TO ACHIEVE